

เอกสารสรุปสถาปัตยกรรมและรายละเอียดของระบบ

TUH CHATBOT AI v2

(ระบบแชทบอทข้อมูลสวัสดิการและมาตรฐาน ISO โรงพยาบาลธรรมศาสตร์เฉลิมพระเกียรติ)

จัดทำและสรุปโดยผู้เชี่ยวชาญด้านการออกแบบระบบและไดอะแกรม

ภาษาที่ใช้งานในระบบ: Python (FastAPI), Javascript (React), SQL

พัฒนาตามข้อกำหนดของโครงสร้างสถาปัตยกรรม v2 (Modern Architecture)

ปรับปรุงล่าสุด: สิงหาคม 2569

บทสรุปเทคโนโลยีแชทบอทและผู้ใช้งาน (Tech Stack & Actors Summary)

1. สแต็กเทคโนโลยีและเฟรมเวิร์กที่ใช้งานจริงในระบบ (Framework Stack)

- ส่วนติดต่อผู้ใช้งานฝั่งแชทและแอตมิน (Frontend / FN): React, Vite, Tailwind CSS
- ส่วนบริการเซิร์ฟเวอร์และตรรกะระบบ (Backend / BN): FastAPI, Uvicorn (ASGI Server), SQLAlchemy (async ORM)
- ฐานข้อมูลหลักฝั่งสัมพันธ์ (Relational SQL Database): TiDB Cloud MySQL (เกตเวย์คลาวด์พอร์ต 4000 แบบ SSL)
- ฐานข้อมูลเวกเตอร์และคำสำคัญ (Vector/Lexical Database): FAISS (Facebook AI Similarity Search) เวกเตอร์อินเด็กซ์ และ BM25 Lexical Search Index จัดทำดัชนีผ่านไลบรารีสกัดท่อนข้อความ/ตารางด้วย BGE-M3 Embeddings

2. กลุ่มผู้เกี่ยวข้องและผู้ใช้งานระบบแชทบอท (System Actors)

- บุคลากรทั่วไปของโรงพยาบาลธรรมศาสตร์เฉลิมพระเกียรติ: ค้นหาถาม-ตอบสวัสดิการหรือคู่มือมาตรฐาน ISO ผ่านหน้าเว็บแชทบอท (Web 1 Chat UI) ได้อย่างสะดวกโดยไม่จำเป็นต้องล็อกอินเข้าระบบ
- แอดมิน (Admin): ล็อกอินเข้าระบบหลังบ้าน (Web 2) เพื่ออัปโหลดไฟล์คู่มือสวัสดิการในรูปแบบ PDF, ตรวจสอบและแก้ไขข้อความท่อนความรู้ (Chunks) รวมถึงข้อมูลในตารางสวัสดิการ และวิเคราะห์ประวัติสถิติคำถามการตอบกลับ
- ซุปเปอร์แอดมิน (Super Admin): มีสิทธิ์การทำงานเหมือนแอดมินทุกประการ เพิ่มเติมคือสามารถเปิดบัญชีแอดมินรายใหม่ได้โดยตรงผ่านระบบ และสามารถปรับแต่งตั้งค่าโมเดล AI (LLM API Key, System Prompt, Parameter) ได้ทันที

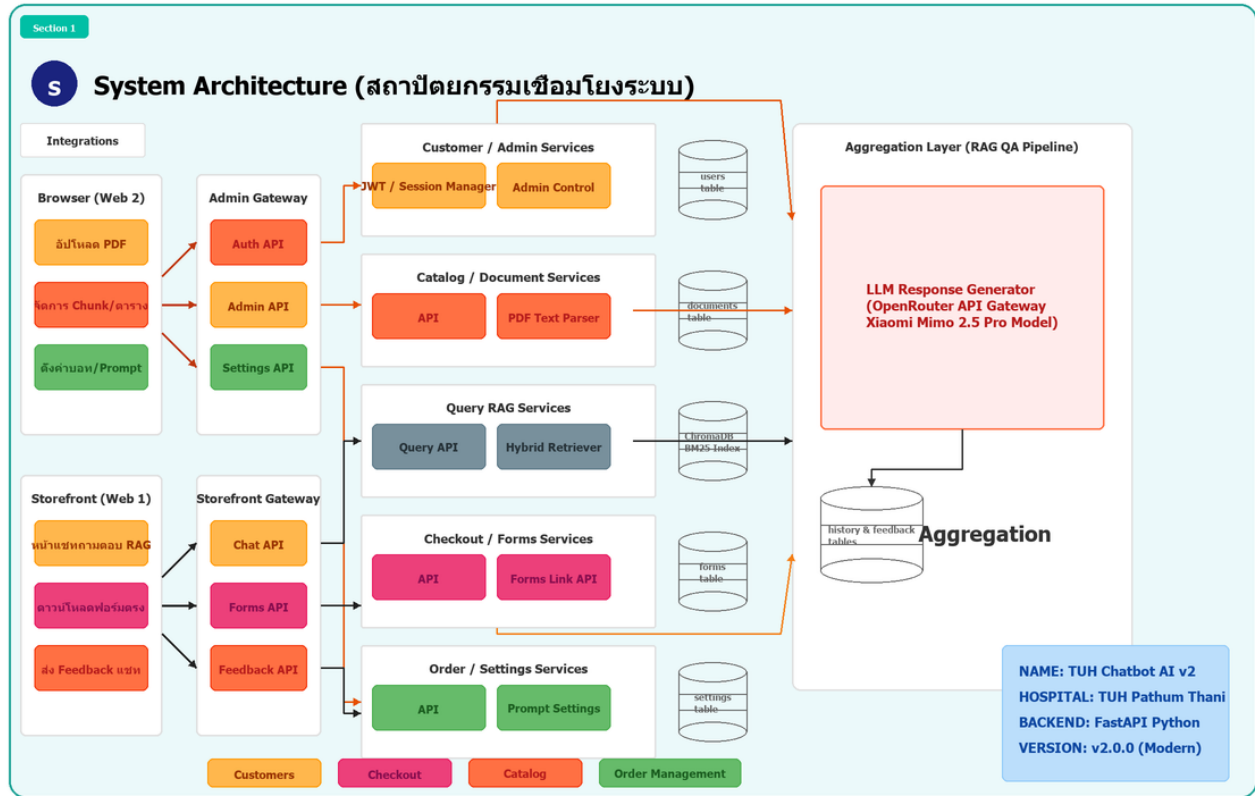
3. สถาปัตยกรรมกระบวนการสืบค้นและตอบกลับอัจฉริยะ (Hybrid RAG Mechanism)

ระบบแชทบอทนี้ทำงานด้วยสถาปัตยกรรม Retrieval-Augmented Generation (RAG) แบบก้าวหน้า โดยมีกระบวนการดังนี้:

1. คัดกรองความสุภาพ (Profanity Filter) และเช็คแชททักทายทั่วไป (Chit-chat Filter) เพื่อการตอบสนองที่เหมาะสมและประหยัด API คอล
2. ค้นหาแบบไฮบริด (Hybrid Search) โดยสืบค้นเชิงความหมายผ่าน FAISS เวกเตอร์ควบคู่กับการค้นหารูปแบบคำสำคัญ Lexical ผ่าน BM25
3. นำคะแนนผลลัพธ์มาผสานและเรียงลำดับใหม่ด้วยสูตร Weighted Reciprocal Rank Fusion (RRF) เพื่อให้ได้เนื้อหาที่ใกล้เคียงที่สุด
4. ส่งข้อมูลบริบท (Context) รวมตารางและข้อความ ไปให้ LLM API (Xiaomi MIMO 2.5 Pro) ผ่าน OpenRouter เพื่อตอบเป็นภาษาไทยอย่างธรรมชาติ

1. System Architecture (สถาปัตยกรรมระบบภาพรวม)

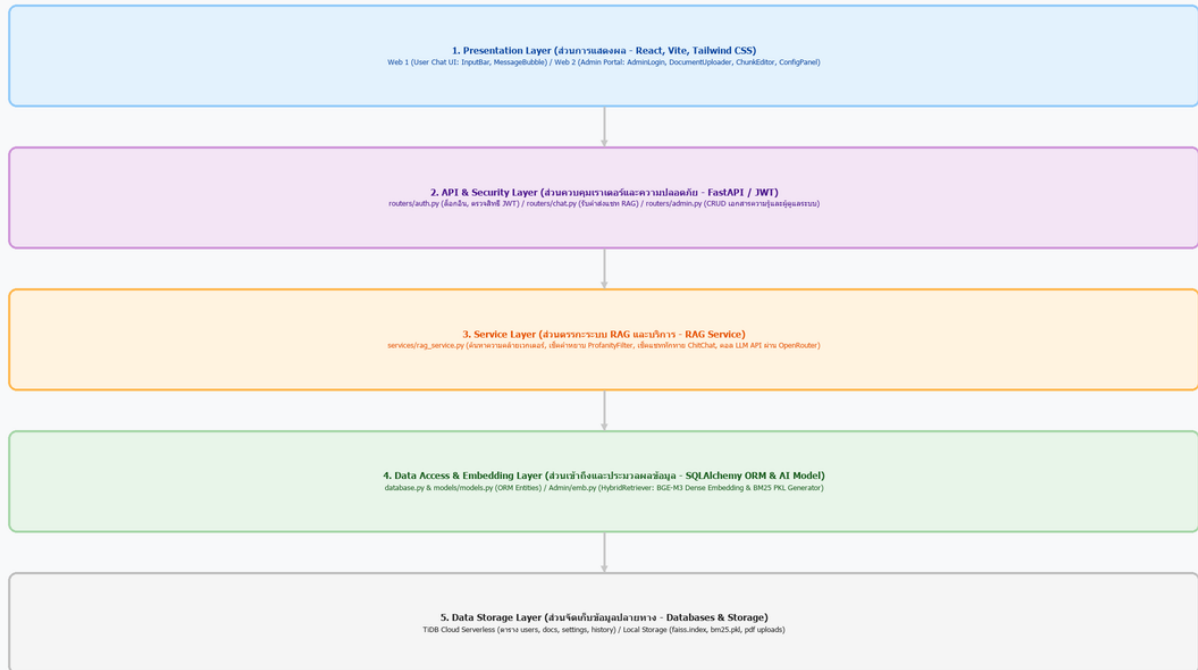
แสดงการเชื่อมต่อระหว่างเบราว์เซอร์ผู้ใช้งานและแอดมิน ผ่านระบบเครือข่ายส่งคำขอไปยังเซิร์ฟเวอร์ FastAPI Backend ซึ่งทำหน้าที่ประสานงานระหว่างฐานข้อมูลความรู้ TiDB Cloud MySQL และดัชนีคำค้นหา FAISS ในรูปแบบการประมวลผล RAG



2. Software Architecture (โครงสร้างซอฟต์แวร์แบบแบ่งชั้นเลเยอร์)

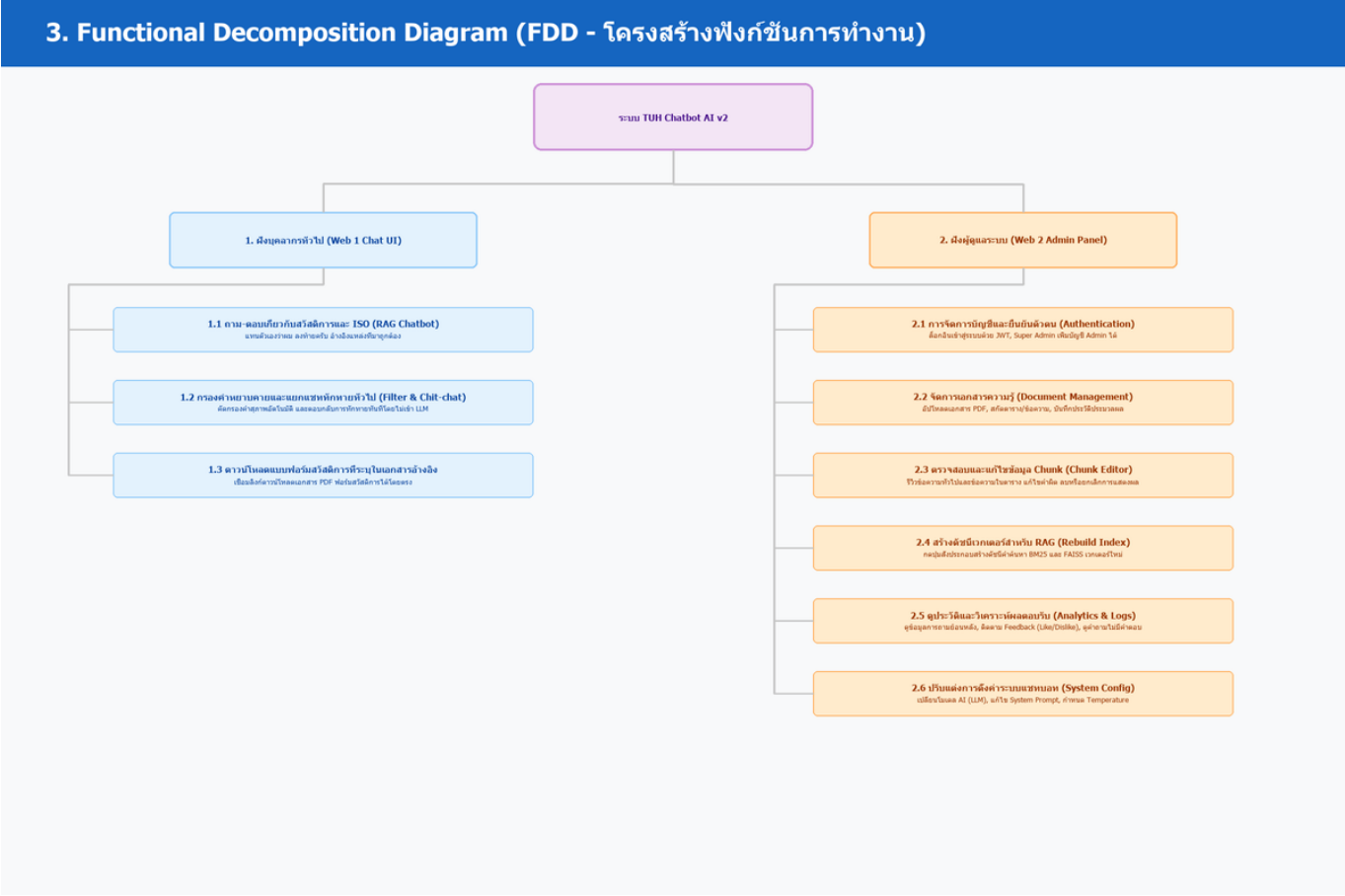
แสดงโครงสร้างการจัดแบ่งโมดูลซอฟต์แวร์ออกเป็น 5 ชั้นอย่างมีระเบียบ ประกอบด้วย Presentation Layer (React), API Layer (FastAPI Routers), Service Layer (RAG Logic), Data Access Layer (SQLAlchemy ORM) และ Storage Layer (TiDB & Local Index)

2. Software Architecture (โครงสร้างซอฟต์แวร์แบบแบ่งเลเยอร์)



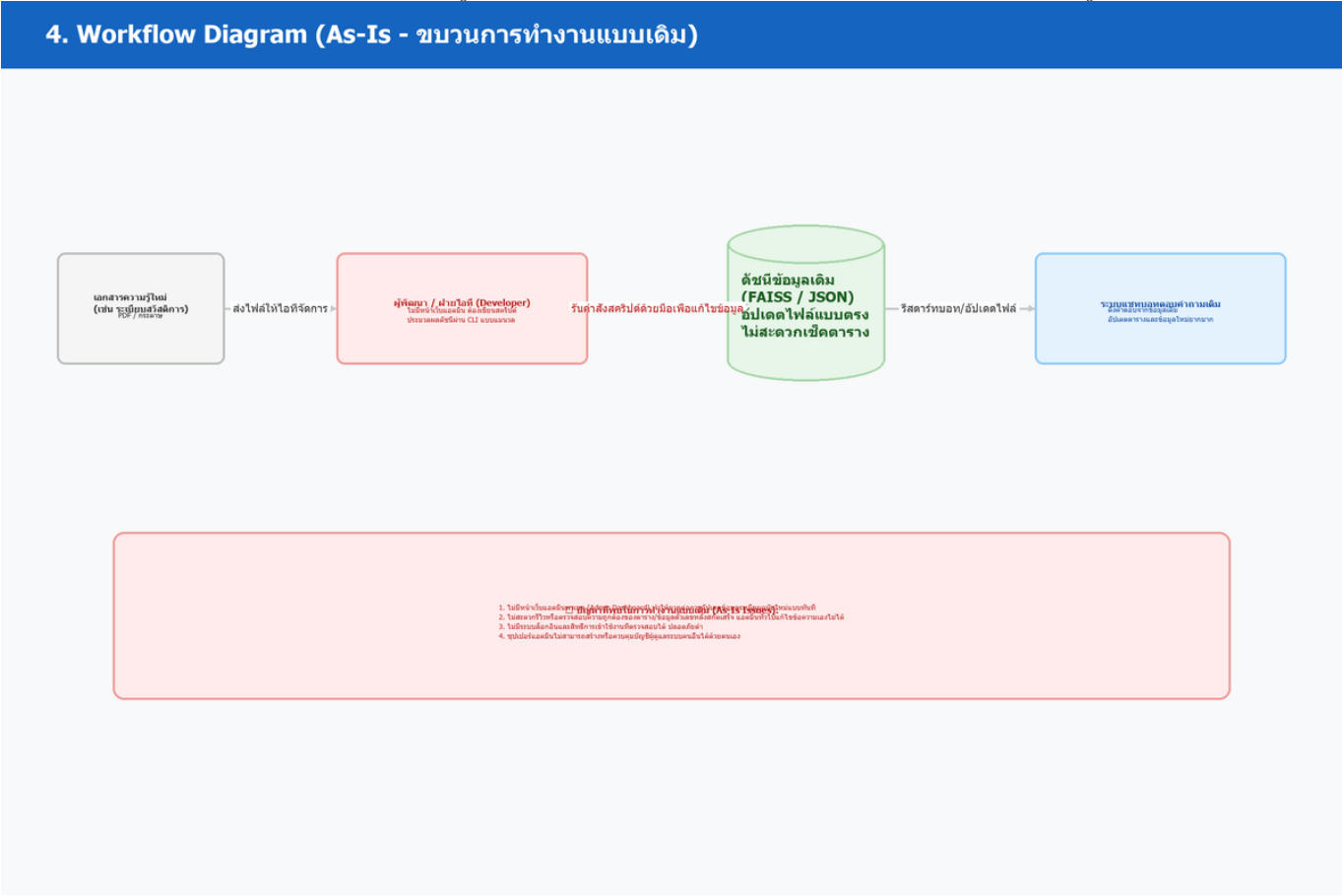
3. Functional Decomposition Diagram (FDD - โครงสร้างฟังก์ชันระบบ)

แสดงการจำแนกฟังก์ชันหน้าที่หลักของระบบตามกลุ่มผู้ใช้งาน โดยผู้ใช้งานทั่วไปเน้นที่การถามตอบสวัสดิการและดาวน์โหลดแบบฟอร์ม ในขณะที่ผู้ดูแลระบบเน้นที่การจัดการควบคุมและจัดการสิทธิ์แอปพลิเคชัน ตรวจสอบรายการ ปรับการตั้งค่าบอท และสถิติประวัติอย่างละเอียด



4. Workflow Diagram - As-Is (ขบวนการอัปเดตข้อมูลแบบเดิม)

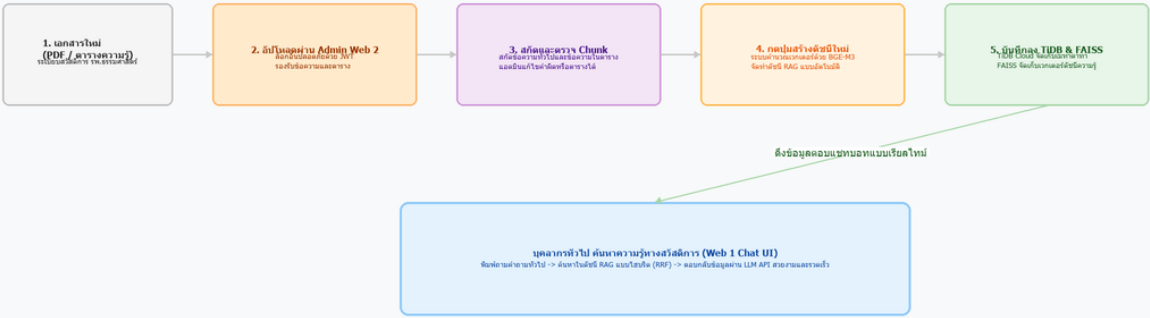
แสดงกระบวนการทำงานในอดีตซึ่งไม่มีหน้าเว็บฝั่งแอดมิน อำนาจและวิธีอัปเดตข้อมูลความรู้ใหม่ต้องทำผ่านไอที/โปรแกรมเมอร์โดยเขียนสคริปต์หรืออิมพอร์ตแมนวลตรงผ่าน CLI ส่งผลให้แก้ไขข้อมูลตารางยาก ลำบาก และไม่มีความปลอดภัยในการตรวจสอบความถูกต้อง



5. Workflow Diagram - To-Be (ขบวนการอัปเดตข้อมูลแบบใหม่ผ่านเว็บแอดมิน)

แสดงกระบวนการทำงานใหม่ที่นำเสนอ
แอดมินสามารถอัปเดตเอกสารสวัสดิการและตรวจสอบความถูกต้องของตาราง/ข้อความก่อนความรู้ผ่านหน้าเว็บแอดมินได้โดยตรง

5. Workflow Diagram (To-Be - ขบวนการทำงานที่นำเสนอใหม่)

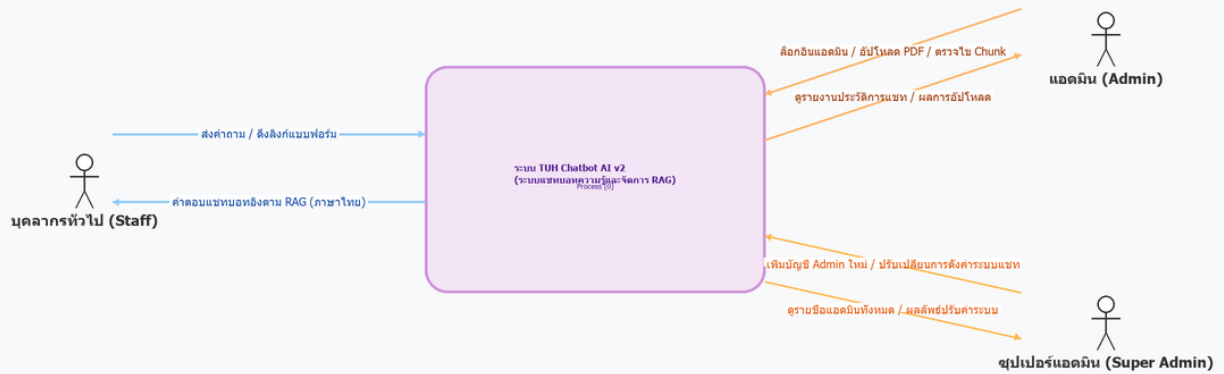


1. ระบบเอกสาร (Web 2) อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ
2. อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ
3. อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ
4. อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ
5. อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ อัปเดตเอกสารสวัสดิการ

6. Data Flow Diagram Level 0 (Context Diagram - แผนผังระดับสูงสุด)

แสดงกระแสข้อมูลระดับบริบทภายนอกเชื่อมโยงกับระบบแชทบอท โดยบุคลากรทั่วไปส่งคำถามและรับคำตอบตอบกลับ แอดมินถือกรีนจัดการ PDF และตรวจแก้ข้อมูล และซูเปอร์แอดมินทำหน้าที่จัดการบัญชีผู้ใช้งานแอดมินเพิ่มเติมด้วยความโปร่งใสและตรวจสอบได้

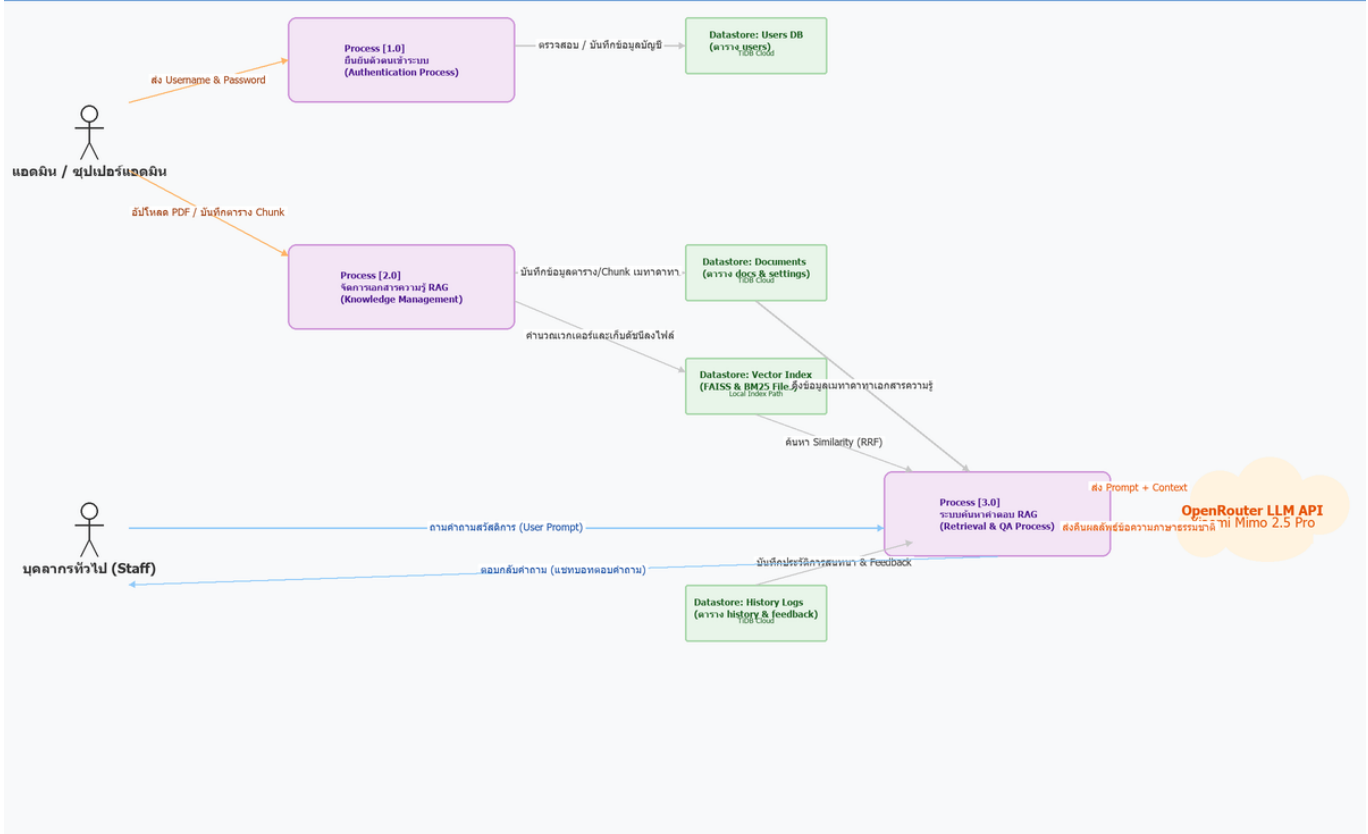
6. DFD Level 0 (Context Diagram - แผนผังระดับบริบทสูงสุด)



7. Data Flow Diagram Level 1 & 2 (แผนผังกระแสข้อมูลระดับย่อย)

แสดงรายละเอียดกระแสข้อมูลไหลผ่านกระบวนการหลัก 3 กระบวนการ ประกอบด้วย การตรวจสอบสิทธิ์การใช้งานแอดมิน (Process 1.0), การจัดการสกัดไฟล์ความรู้เอกสารและสร้างเวกเตอร์ (Process 2.0) และกระบวนการสืบค้น Hybrid RAG ค้นคำตอบอัจฉริยะ (Process 3.0)

7. DFD Level 1 (แผนผังกระแสข้อมูลระดับ 1 แยกตามฟังก์ชัน)



8. Use Case Diagram (แผนภาพกรณีการใช้งานระบบ)

แสดง Use Cases ทั้งหมดของระบบภายใต้ขอบเขตแอปพลิเคชัน (System Boundary) เชื่อมโยงกับ Actors ทั้ง 3 กลุ่ม เช่น การถามตอบสวัสดิการ, อัปโหลดเอกสาร PDF, แก้ไขตารางสวัสดิการ, ปรับปรุงการตั้งค่า prompt/LLM และการเพิ่มสมาชิกผู้ใช้งานในระบบ

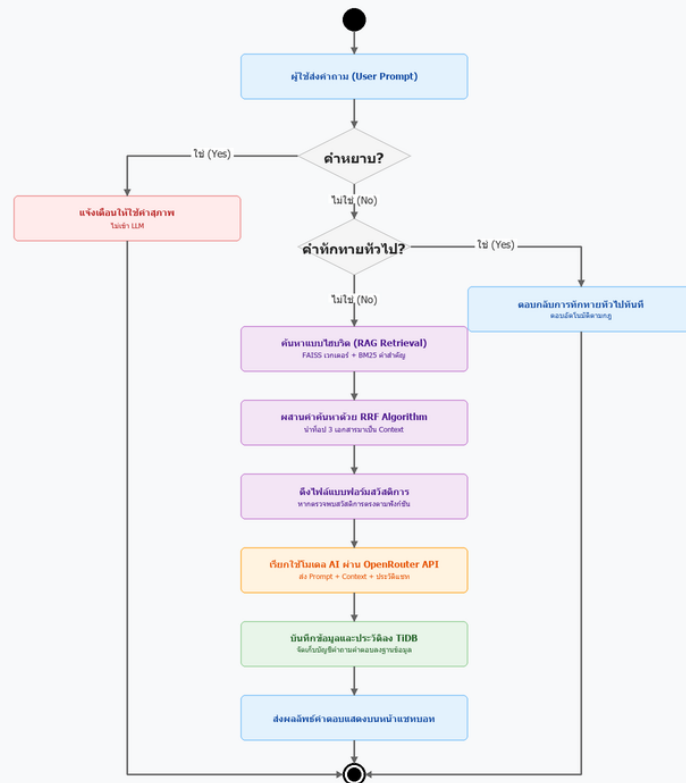
8. Use Case Diagram (แผนภาพกรณีการใช้งานระบบ)



9. Activity Diagram (แผนภาพแสดงกิจกรรมขั้นตอน RAG)

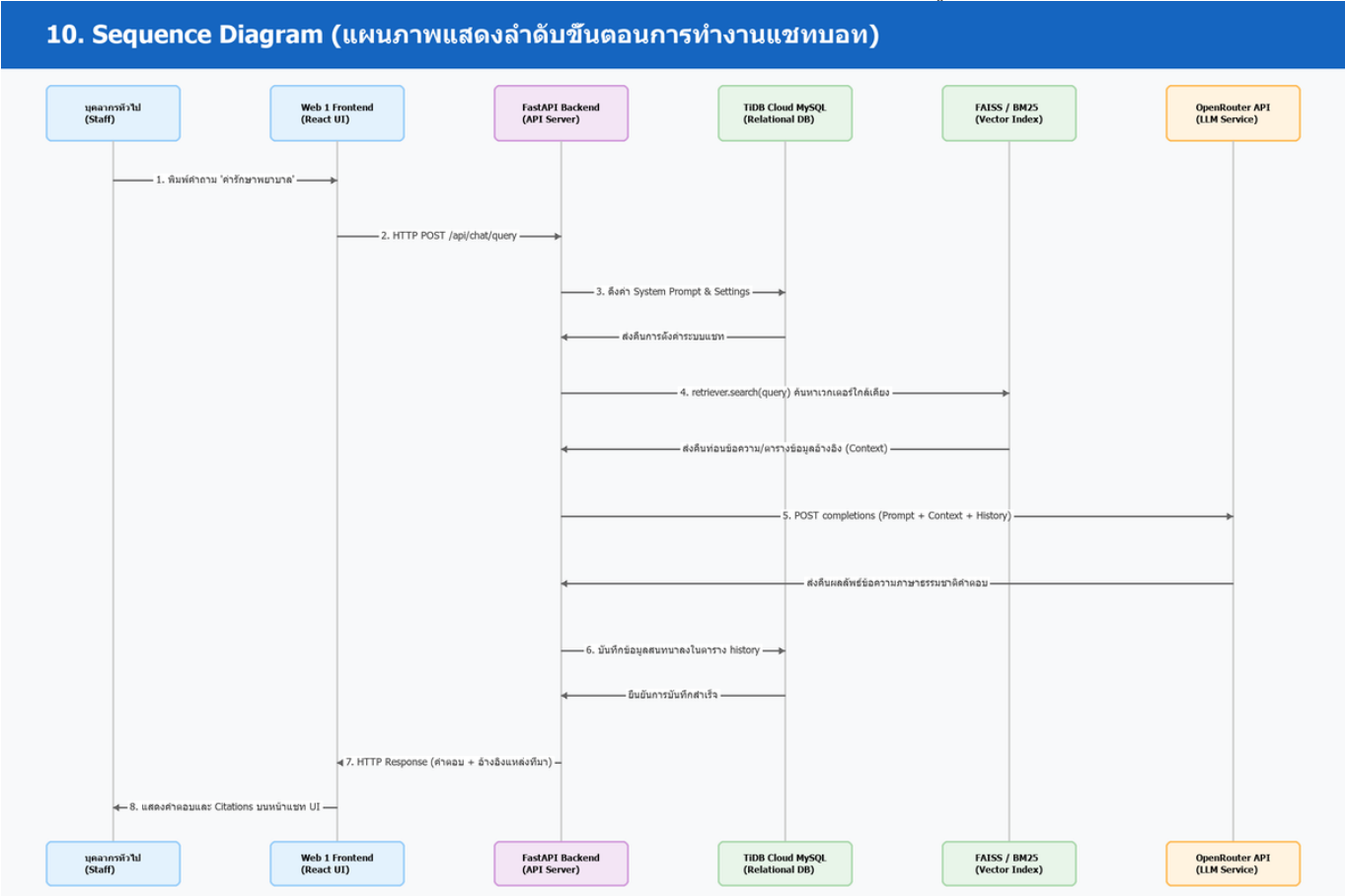
แสดงกิจกรรมและเงื่อนไขการทำงานอย่างละเอียดของระบบ RAG เริ่มจากการรับอินพุตคำถามของผู้ใช้, ตรวจสอบคำหยาบ Profanity, ตรวจสอบแชททักทาย Chit-chat, ค้นหาแบบไฮบริด (FAISS + BM25) ผสานด้วย RRF, ดึงลิงก์แบบฟอร์มสวัสดิการที่เกี่ยวข้อง และส่งไปยัง LLM AP

9. Activity Diagram (แผนภาพกิจกรรมการค้นหา RAG และสร้างคำตอบ)



10. Sequence Diagram (แผนภาพแสดงลำดับเวลาการตอบแชทบอท)

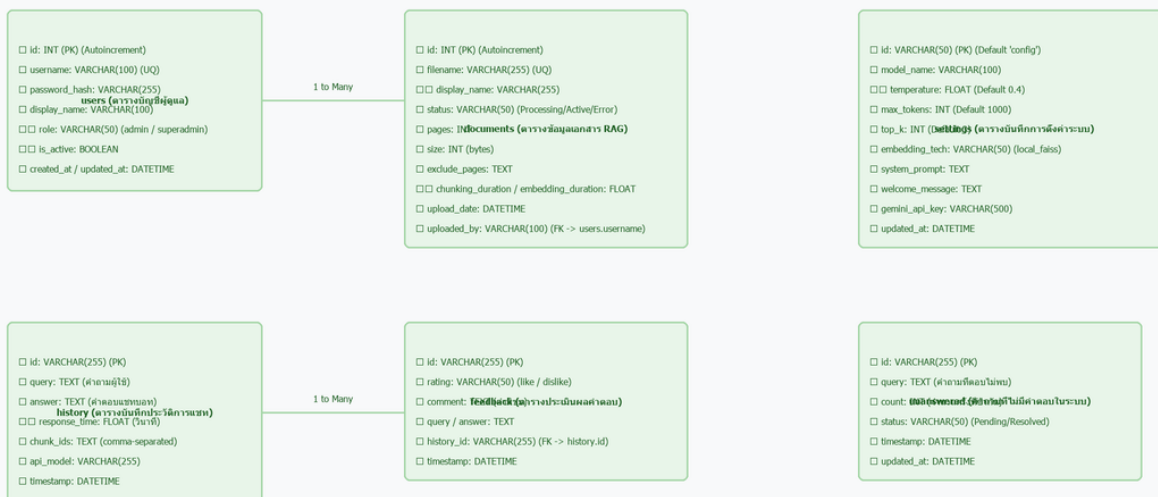
แสดงการปฏิสัมพันธ์แบบซิงโครนัสตามเส้นแกนเวลา (Lifelines) ระหว่างผู้ใช้งานฝั่งแชทบอท, React UI, FastAPI Backend API, ฐานข้อมูล TiDB Cloud, คีย์ไฟล์ FAISS เวกเตอร์ดัชนี และ OpenRouter LLM Gateway จนกระทั่งตอบผลลัพธ์กลับสู่หน้าจออย่างเสร็จสิ้น



11. Entity-Relationship Diagram (ER Diagram - แผนภาพความสัมพันธ์ฐานข้อมูล)

แสดงโครงสร้างตารางข้อมูลในระบบ (Database Schema) บนฐานข้อมูล TiDB Cloud MySQL ประกอบด้วยตาราง users (เก็บบัญชี), documents (เอกสาร), settings (การตั้งค่า), history (ประวัติแชท), feedback (ผลการดไลก์), unanswered และตารางแบบฟอร์มสวัสดิการ

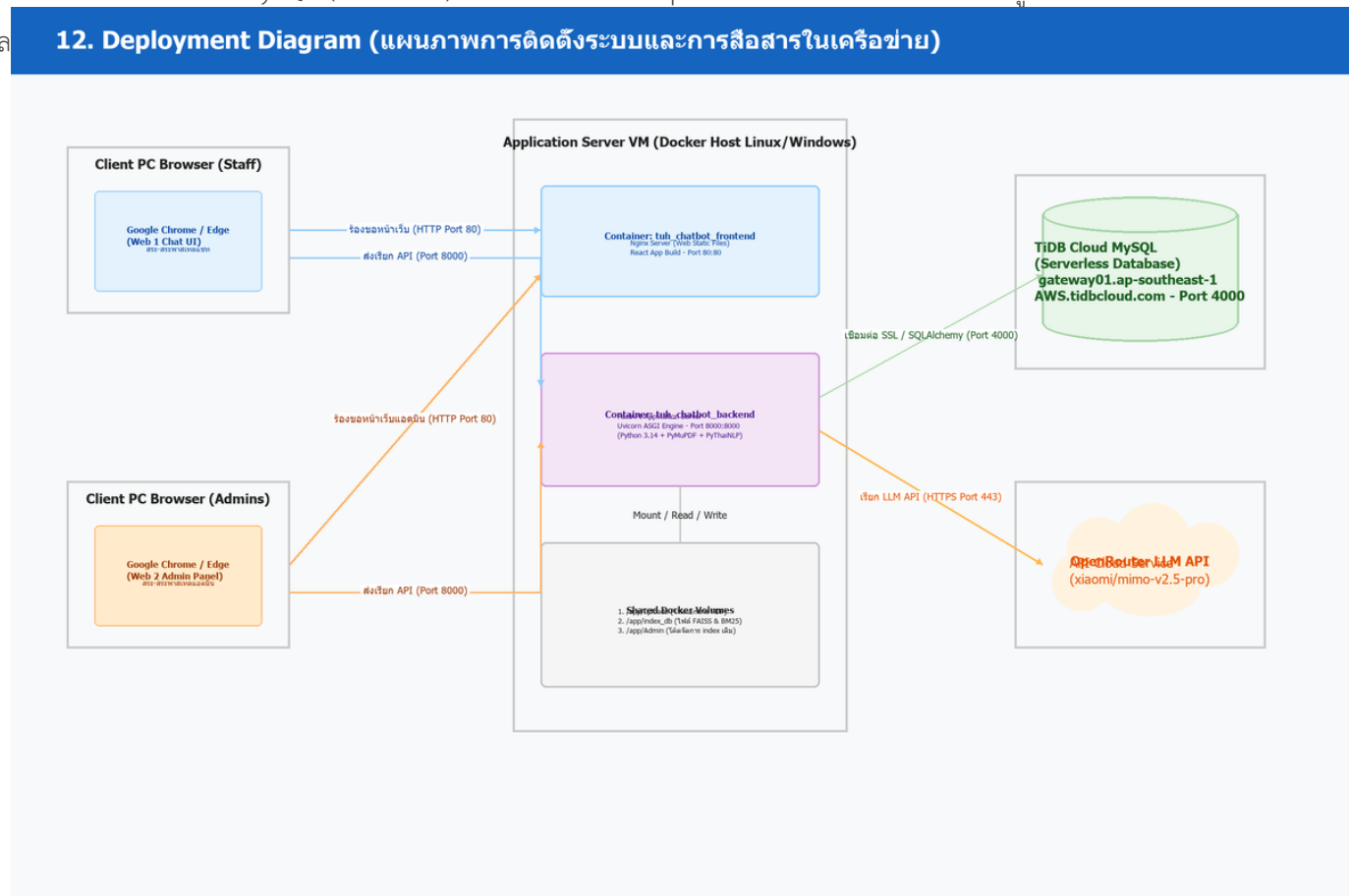
11. Entity-Relationship Diagram (ER Diagram - แผนภาพฐานข้อมูล)



12. Deployment Diagram (แผนภาพการติดตั้งอุปกรณ์และเครือข่าย)

แสดงการติดตั้งระบบแบบ Docker Compose โดยรัน Web Nginx Port 80 และ FastAPI API Port 8000 บนเซิร์ฟเวอร์ พร้อมเชื่อมต่อไปยัง TiDB Cloud Serverless MySQL (Port 4000) และคลาวด์เกตเวย์ OpenRouter LLM API เชื่อมโยงข้อมูลผ่านการเข้ารหัส SSL และ HTTPS

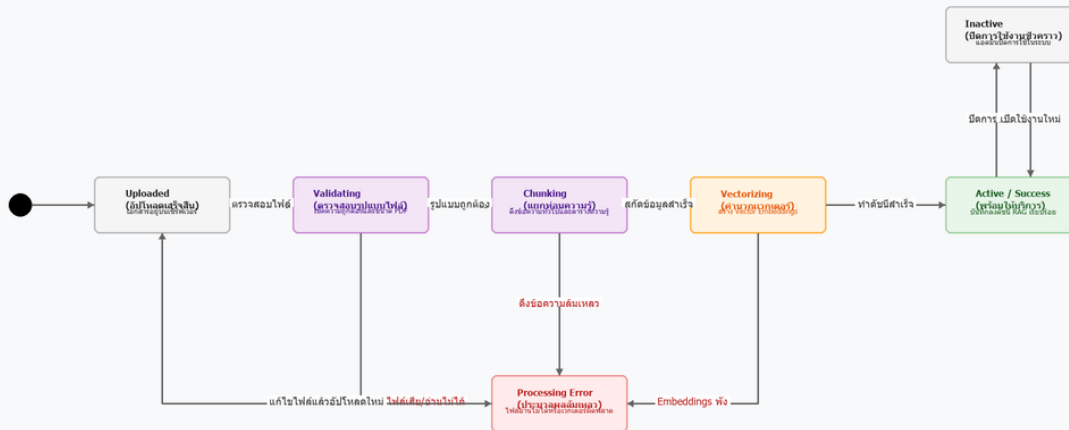
ปล **12. Deployment Diagram (แผนภาพการติดตั้งระบบและการสื่อสารในเครือข่าย)**



13. State Machine Diagram (แผนภาพสถานะวงจรชีวิตของเอกสารสวัสดีการ)

แสดงลำดับการเปลี่ยนแปลงสถานะ (Lifecycle States) ของไฟล์เอกสารความรู้ที่อัปโหลดเข้าระบบ RAG ตั้งแต่สถานะอัปโหลด (Uploaded), กำลังตรวจสอบรูปแบบ (Validating), กำลังสกัดท่อนข้อความ (Chunking), กำลังแปลงเวกเตอร์ (Vectorizing) จนกระทั่งสำเร็จใช้งาน (Active)

13. State Machine Diagram (แผนภาพสถานะวงจรชีวิตของเอกสารความรู้)



14. Class / Component Diagram (แผนภาพคลาสโครงสร้างและโมดูลสถาปัตยกรรม)

แสดงองค์ประกอบเชิงคลาส (UML Class Diagram) ของ FastAPI Backend เช่น Database, Security, HybridRetriever, RAGService รวมถึงการแมปปิ้งเราเตอร์ (Routes) และโมเดลข้อมูล SQLAlchemy Entities ที่ทำงานสอดประสานกันภายในระบบอย่างเป็นระบบ

14. Class / Component Diagram (แผนภาพคลาสและโมดูลสถาปัตยกรรม)

